1)	Discuss base and limit register concepts using a proper flowchart.

Ans

The **base and limit registers** are used in an operating system to **protect memory**.

- The **base register** stores the **starting address** of a process.
    - The **limit register** stores the **size (maximum range)** of that process.

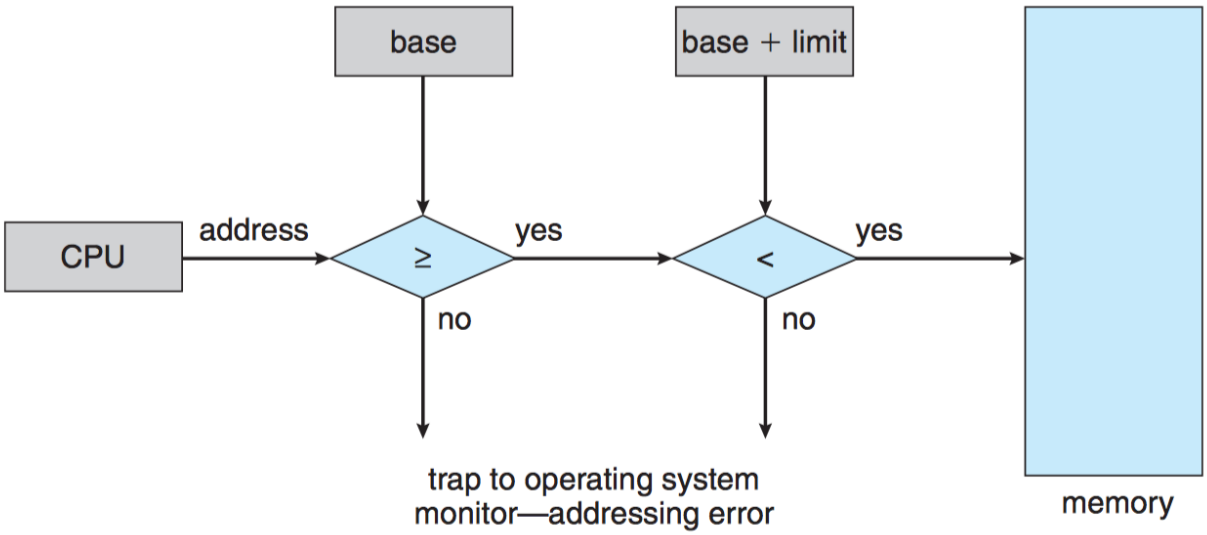


Figure 8.2 Hardware address protection with base and limit registers.

---

1. Assume that there are 5 processes. Consider the following data table and formulate banker algorithm to find out the safe sequence. Here, Max Instances of Resource Type A = 3, Max Instances of Resource Type B = 17, Max Instances of Resource Type C = 16, Max Instances of Resource Type D = 12

Process	Allocation Matrix				Max. Need Matrix			
	A	B	C	D	A	B	C	D
P1	0	1	1	0	0	2	1	0
P2	1	2	3	1	1	6	5	2
P3	1	3	6	5	2	3	6	6
P4	0	6	3	2	0	6	5	2
P5	0	0	1	4	0	6	5	6

Q-1

Process	Max Allocation				Max Need				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P_1	0	1	1	0	0	1	2	0	0	1	1	0
P_2	1	2	3	1	1	6	5	2	0	4	2	1
P_3	1	3	6	5	2	3	6	6	1	0	0	1
P_4	0	6	3	2	0	6	5	2	0	0	2	0
P_5	0	0	1	4	0	6	5	6	0	6	4	2
	2	12	14	12								

Max-Allocation

A	B	C	D	
3	17	16	12	
2	12	14	12	
1	5	2	0	
0	1	1	0	$\Rightarrow P_1$
1	6	3	0	
0	6	3	2	$\Rightarrow P_4$
1	12	6	2	
1	2	3	1	$\Rightarrow P_2$
2	14	9	3	
1	3	6	5	$\Rightarrow P_3$
3	17	15	8	
0	0	1	4	$= P_5$
3	17	16	12	

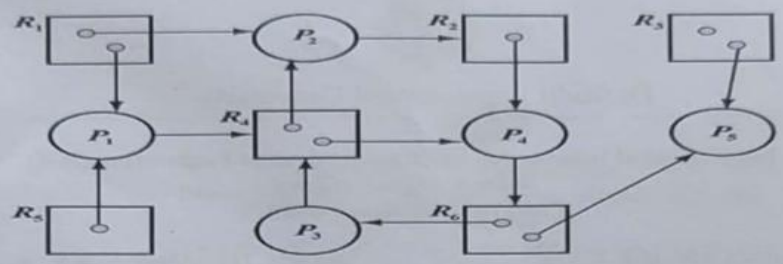
∴ Safe sequence -

P1 -> P4 -> P2 -> P3 -> P5

(Ans)

---

5)



[5]

CLO3
C5

Conclude whether there is a deadlock in this RAG(Resource Allocation Graph) or not.

Process	In: Allocation						Out: Request					
	R1	R2	R3	R4	R5	RC	R1	R2	R3	R4	R5	RC
P1	1	0	0	0	1	0	0	0	0	1	0	0
P2	1	0	0	1	0	0	0	1	0	0	0	0
P3	0	0	0	0	0	1	0	0	0	1	0	0
P4	0	1	0	1	0	0	0	0	0	0	0	1
P5	0	0	1	0	0	1	0	0	0	0	0	0
	2	1	1	2	1	2						

Available

	0	0	0	0	0	0	
(+)	0	0	1	0	0	1	-> P5
	0	0	1	0	0	1	
(+)	0	1	0	1	0	0	-> P4
	0	1	1	1	0	1	
(+)	1	0	0	0	1	0	-> P1
	1	1	1	1	1	1	
(+)	1	0	0	1	0	0	-> P2
	2	1	1	2	1	1	
(+)	0	0	0	0	0	1	-> P3
	2	1	1	2	1	2	

Safe sequence -

P5 -> P4 -> P1 -> P2 -> P3

No deadlock.

Define deadlock. Assess the ways to prevent deadlock.

=

A deadlock is a situation in an operating system where a group of processes are blocked because each process is holding at least one resource and waiting for another resource that is held by another process in the group. As a result, none of the processes can continue execution.

Ways to Prevent Deadlock:

Deadlock prevention works by ensuring that at least one of the four necessary deadlock conditions cannot occur.

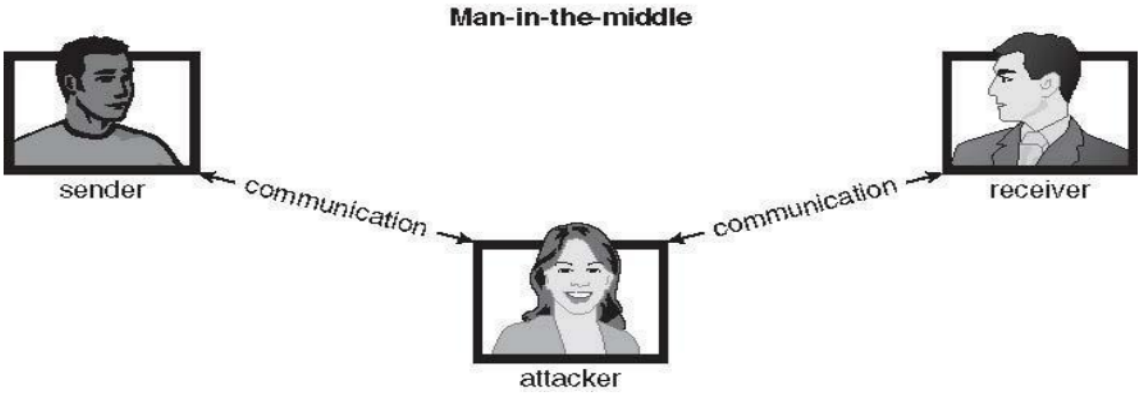
1. **Mutual Exclusion Prevention**
    - Use sharable resources whenever possible.
    - Deadlock cannot occur if resources can be shared among multiple processes.
    - However, some resources are naturally non-shareable.
2. **Hold and Wait Prevention**
    - A process must request all required resources at once before execution starts.
    - Alternatively, a process can request resources only when it holds none.
    - This reduces the chance of waiting for additional resources.
    - Disadvantage: low resource utilization and possible starvation.
3. **No Preemption Prevention**
    - If a process requests a resource that is unavailable, it must release all currently held resources.
    - Released resources are reallocated later when all needed resources become available.
    - This prevents indefinite waiting.
4. **Circular Wait Prevention**
    - Assign a fixed ordering to all resource types.
    - Processes must request resources in increasing order only.
    - This removes circular dependency among processes.

4) a) There are many security violation methods such as replay attack, man in th middle attack, session hijacking, masquerading etc. Illustrate man in th middle and masquerading attack.

a)

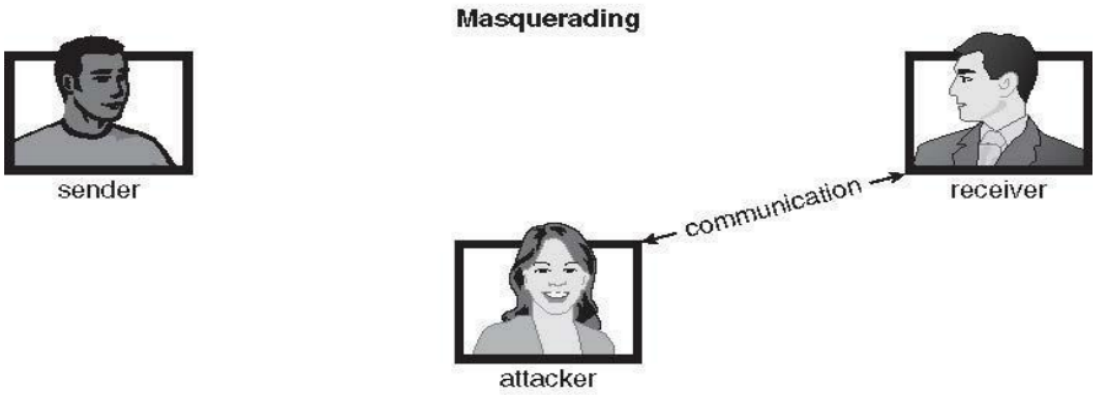
Man-in-the-Middle (MITM) Attack:

In a man-in-the-middle attack, an intruder secretly sits between the sender and receiver during communication. The attacker intercepts, reads, or modifies the data while pretending to each side that they are communicating directly with the other.



Masquerading Attack:

Masquerading is a security violation where an unauthorized user pretends to be an authorized user to gain access to a system or resources. This is mainly a breach of authentication.



Define swapping. Explain the concept of swapping using a schematic view diagram.

Swapping is a memory management technique in operating systems that moves inactive processes from RAM to secondary storage (swap space) and vice versa, freeing up RAM for active processes.

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
- Backing store – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- Roll out, roll in – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows) System maintains a ready queue of ready-to-run processes which have memory images on disk

Schematic View of Swapping